

Python Programming Tasks

1. Safecracker: Palindrome and Digit Sum

You're a safecracker. The vault code is a number, and it only opens if the number is a palindrome AND divisible by the sum of its own digits.

Write `can_crack(n)` that returns True/False.

Input / Output :

Input: 121

Sum of digits = $1+2+1 = 4 \rightarrow 121 \% 4 \neq 0 \rightarrow$ Palindrome but doesn't divide evenly

Output: False

Input: 121 $\rightarrow$ wait, try 12321

Sum of digits = 9 $\rightarrow 12321 \% 9 == 0$ and it's a palindrome

Output: True

Don't convert the number to a string to check the palindrome — do it purely with arithmetic (`%` and `//`).

2. Most frequent word program

Given a paragraph of text (multiple sentences, mixed case, punctuation), find:

- The most frequently occurring word (case-insensitive, ignore punctuation)
- The longest word that appears more than once

Input / Output:

Input: "The cat sat. The CAT ran! The dog barked, but the cat slept."

Output:

Most frequent word: 'the' (4 times)

Longest repeated word: 'cat'.

3. Staircase Climbing: Recursion and Memoization

A person climbing a staircase can take 1, 2, or 3 steps at a time. Write `count_ways(n)` that returns how many distinct ways they can climb a staircase of `n` steps.

Input: `n = 4`

Output: 7

Solve it two ways in the same file:

1. `count_ways_recursive(n)` — plain recursion
2. `count_ways_memo(n)` — same logic, but memoized (use `functools.lru_cache` or a manual dictionary cache)

Print the time taken for `n = 30` using both versions with the `time` module, and comment on what you observe.

4. Collatz conjecture

The Collatz Conjecture: starting from any positive integer `n`,

- if `n` is even, next term is `n / 2`
- if `n` is odd, next term is `3n + 1`
- repeat until you reach 1.

Write a recursive function `collatz_steps(n)` that returns the number of steps to reach 1, and memoize results so repeated calls (e.g., a loop calling this for every number from 1 to 10000) don't recompute known chains.

Input: 27

Output: 111 steps

Print which number between 1 and 10,000 produces the longest chain, and how many steps it takes.

5. Library management system

Build a small library system:

- A base class `Item` with private attributes `__title` and `__is_borrowed` (default `False`), and methods `borrow()`, `return_item()`, and `__str__()` (so `print()` shows something readable).
- Two subclasses: `Book` (adds `author`, `pages`) and `DVD` (adds `duration_minutes`). Both override `__str__()` to show their own details (polymorphism).
- A `Library` class that holds a list of `Item` objects with methods:
 - `add_item(item)`
 - `borrow_item(title)` — finds by title, borrows if available, else prints a message
 - `show_available()` — prints all items not currently borrowed, using each item's own `__str__()`

Sample interaction:

```
lib = Library()
lib.add_item(Book("Atomic Habits", "James Clear", 320))
lib.add_item(DVD("Inception", 148))
lib.borrow_item("Inception")
lib.show_available()
```

Output:

Available Items:

Book: 'Atomic Habits' by James Clear (320 pages)

6. Circle Class

Define a `Circle` class allowing to create a circle $C(O, r)$ with center $O(a, b)$ and radius r using the constructor:

```
def __init__(self, a, b, r):
    self.a = a
    self.b = b
    self.r = r
```

Define an `Area()` method of the class which calculates the area of the circle.

Define a `Perimeter()` method of the class which allows you to calculate the perimeter of the circle.

Define a testBelongs() method of the class which allows to test whether a point A(x, y) belongs to the circle C(O, r) or not.

7. Computation Class

Create a Computation class with a default constructor (without parameters) allowing to perform various calculations on integers numbers.

Create a method called Factorial() which allows to calculate the factorial of an integer. Test the method by instantiating the class.

Create a method called Sum() allowing to calculate the sum of the first n integers:

$$1+2+3+\dots+n$$

Create a method called testPrim() in the Computation class to test the primality of a given integer. Test this method.

Create a method called testPrims() allowing to test if two numbers are prime between them.

8. Inheritance

Define a base class Person with attributes name and age. Create a constructor to initialize these attributes and a method displayPerson() to display the details.

Define a class Student that inherits from Person. Add attributes rollNo and course. Create a constructor to initialize all the attributes and a method displayStudent() to display the student details.

Define a class Teacher that inherits from Person. Add attributes employeeId and subject. Create a constructor to initialize all the attributes and a method displayTeacher() to display the teacher details.

Create one object of the Student class and one object of the Teacher class.

Display the details of both the student and teacher.

Demonstrate inheritance by accessing the displayPerson() method of the parent class from both child classes.

9. Anagram Checker

Write a Python function `is_anagram(word1, word2)` that checks if two words are anagrams.

An anagram is a word or phrase formed by rearranging the letters of another word, using all the original letters exactly once.

Input / Output:

Enter first word: listen

Enter second word: silent

Output: Yes

Enter first word: apple

Enter second word: pale

Output: No

10. Hill Number Check

Write a function `is_hill_number(n)` that checks if a number is a hill number (digits first strictly increase, then strictly decrease). Accept a number from the user and print "Yes" or "No".

Input / Output:

Enter a number: 12321

Output: Yes

Enter a number: 12345

Output: No

Enter a number: 54321

Output: No

Bonus Tasks

1. Stack

Implement a class Stack with methods:

push(x) → adds an element

pop() → removes the top element

peek() → shows the top element

Accept user input to perform stack operations and print results. Do not use the inbuilt functions .pop and .push

2. Singly Linked List

Implement a class LinkedList with methods:

insert_end(x) → inserts a node at the end

delete_end() → deletes a node from the end

display() → prints the list Accept user input to insert and delete values, and display the linked lis